

# AgentScope Java

用 **Java** 构建生产级 **AI** 智能体

---

## 什么是 AgentScope Java?

AgentScope Java 是一个面向智能体的编程框架，用于构建 LLM 驱动的应用程序。它提供了创建智能体所需的一切：ReAct 推理、工具调用、内存管理、多智能体协作等。

## 核心亮点

### 智能体自主，全程可控

AgentScope 采用 ReAct（推理-行动）范式，使智能体能够自主规划和执行复杂任务。与传统的工作流方法不同，ReAct 智能体能够动态决定使用哪些工具以及何时使用，实时适应不断变化的需求。

然而，在生产环境中，没有控制的自主性是一种隐患。AgentScope 提供了全面的运行时干预机制：

- **安全中断** - 在任意时刻暂停智能体执行，同时保留完整的上下文和工具状态，支持无数据丢失的无缝恢复
- **优雅取消** - 终止长时间运行或无响应的工具调用，而不会破坏智能体状态，允许立即恢复和重定向
- **人机协作** - 通过 Hook 系统在任何推理步骤注入修正、额外上下文或指导，保持人类对关键决策的监督

## 内置工具

AgentScope 包含生产就绪的工具，解决智能体开发中的常见挑战：

- **PlanNotebook** - 结构化任务管理系统，将复杂目标分解为有序、可追踪的步骤。智能体可以创建、修改、暂停和恢复多个并发计划，确保多步骤工作流的系统化执行。

- **结构化输出** - 自纠错输出解析器，保证类型安全的响应。当 LLM 输出偏离预期格式时，系统会自动检测错误并引导模型生成有效输出，将结果直接映射到 Java POJO，无需手动解析。
- **长期记忆** - 跨会话的持久化内存存储，支持语义搜索。支持自动管理、智能体控制记录或混合模式。支持多租户隔离，满足企业部署中智能体服务多用户的需求。
- **RAG（检索增强生成）** - 与企业知识库无缝集成。支持自托管的基于嵌入的检索和阿里云百炼等托管服务，使智能体响应基于权威数据源。

## 无缝集成

AgentScope 设计为与现有企业基础设施集成，无需大量修改：

- **MCP 协议** - 与任何 MCP 兼容服务器集成，即时扩展智能体能力。连接到不断增长的 MCP 工具和服务生态系统——从文件系统和数据库到 Web 浏览器和代码解释器——无需编写自定义集成代码。
- **A2A 协议** - 通过标准服务发现实现分布式多智能体协作。将智能体能力注册到 Nacos 或类似注册中心，允许智能体像调用微服务一样自然地发现和调用彼此。

## 生产级别

为企业部署需求而构建：

- **高性能** - 基于 Project Reactor 的响应式架构确保非阻塞执行。GraalVM 原生镜像编译实现 200ms 冷启动时间，使 AgentScope 适用于 Serverless 和自动扩缩容环境。
- **安全沙箱** - AgentScope Runtime 为不受信任的工具代码提供隔离的执行环境。包括用于 GUI 自动化、文件系统操作和移动设备交互的预构建沙箱，防止未经授权访问系统资源。
- **可观测性** - 原生集成 OpenTelemetry，实现整个智能体执行管道的分布式追踪。AgentScope Studio 为开发和生产环境提供可视化调试、实时监控和全面的日志记录。

## 系统要求

- JDK 17 或更高版本
- Maven 或 Gradle

## 快速开始

按照以下步骤开始使用 AgentScope Java:

1. [安装](#) - 在您的项目中配置 AgentScope Java
2. [核心概念](#) - 理解核心概念和架构
3. [构建第一个智能体](#) - 创建一个可工作的智能体

## 快速示例

```
import io.agentscope.core.ReActAgent;
import io.agentscope.core.model.DashScopeChatModel;
import io.agentscope.core.message.Msg;

// 创建智能体并内联配置模型
ReActAgent agent = ReActAgent.builder()
    .name("Assistant")
    .sysPrompt("你是一个有帮助的 AI 助手。")
    .model(DashScopeChatModel.builder()
        .apiKey(System.getenv("DASHSCOPE_API_KEY"))
        .modelName("qwen3-max")
        .build())
    .build();

Msg response = agent.call(Msg.builder()
    .textContent("你好! ")
    .build()).block();
System.out.println(response.getTextContent());
```

## 进阶指南

熟悉基础后，可以探索以下功能：

### 模型集成

- [模型集成](#) - 配置不同的 LLM 提供商
- [多模态](#) - 视觉和多模态能力

### 工具与知识

- [工具系统](#) - 使用基于注解的工具注册创建和使用工具
- [MCP](#) - 模型上下文协议支持，实现高级工具集成
- [RAG](#) - 检索增强生成，提供知识增强的响应

- [结构化输出](#) - 类型安全的输出解析，支持自动纠错

## 行为控制

- [钩子系统](#) - 使用事件钩子监控和定制智能体行为
- [内存管理](#) - 管理对话历史和长期内存
- [计划](#) - 复杂多步骤任务的计划管理
- [智能体配置](#) - 高级智能体配置选项

## 多智能体系统

- [管道](#) - 使用顺序和并行执行构建多智能体工作流
- [A2A 协议](#) - Agent2Agent 协议支持
- [状态管理](#) - 跨会话持久化和恢复智能体状态

## 可观测性与调试

- [AgentScope Studio](#) - 可视化调试和监控

# AI 辅助开发

AgentScope 文档支持 [llms.txt](#) 标准，使 Claude Code、Cursor、Windsurf 等 AI 编程助手能够理解 AgentScope API 并生成准确的代码。

Cursor 快速设置：

1. 打开 Cursor 设置 -> Features -> Docs
2. 点击 “+ Add new Doc”
3. 添加 URL: `https://java.agentscope.io/llms-full.txt`

更多工具和详细配置，请参阅 [使用 AI 编程](#)。

## 社区

- GitHub: [agentscope-ai/agentscope-java](#)